SQL Chapter 4 Study Guide: General, Conditional, and Conversion Functions

1. Introduction to Single Row Functions

Single row functions are the workhorses of SQL, operating on individual rows to return exactly one result per row. While Oracle categorizes these into five types— **Character, Number, Date, Conversion, and General** —this guide focuses on the latter two. Mastery of General and Conversion functions is mandatory for any developer aiming to handle NULL values gracefully and transform data for complex reporting.

2. General Functions (Null-Related)

General functions provide the logic necessary to handle NULL values across any data type.

NVL Function

The NVL function replaces a NULL value with a meaningful substitute.

- **Arguments:** It accepts exactly two arguments.
- **Logic:** If the first argument is NULL, it returns the second; if the first is **NOT** NULL, it returns the first.
- **Data Type Rules:**
- If the first argument is Character, the second can be Character, Number, or Date.
- If the first argument is Non-Character (Number or Date), both arguments **must** match.
- **Instructor Insight:** You will frequently encounter ORA-01722: invalid number if you attempt to return a string like 'NA' for a numeric column. You **must** convert the numeric column using TO_CHAR first.
- *Example:* SELECT COMMISSION_PCT, NVL(TO_CHAR(COMMISSION_PCT), 'NA') FROM EMPLOYEES;

NVL2 Function

This function evaluates a value and returns different results based on whether that value is null or not.

- **Arguments:** It takes exactly three arguments.
- **Logic:** If the first argument is **NOT** NULL, it returns the second; if it is NULL, it returns the third.
- **Data Type Rules:** If the second argument is Character, the third can be any type. If the second is Non-Character, the second and third **must** match.
- **Example:** SELECT MANAGER_ID, NVL2(MANAGER_ID, 'EMPLOYEE', 'BOSS') FROM EMPLOYEES;

NULLIF Function

NULLIF compares two expressions for equality.

- **Logic:** Returns NULL if the arguments are equal; returns the first argument if they differ.
- **Mandatory Rule:** Both arguments **must** share the same data type.

- **Professional Application:** Often nested within NVL2 to create "Identical" vs "Different" reports.
- *Example:* SELECT PHONE_NUMBER, NVL2(NULLIF(SUBSTR(PHONE_NUMBER, 4, 1), SUBSTR(PHONE_NUMBER, 8, 1)), 'DIFF.', 'IDENTICAL') FROM EMPLOYEES;

COALESCE Function

An extension of NVL, COALESCE evaluates a list of arguments from left to right.

- **Logic:** Returns the first non-null value in the list. It can take an "infinite" number of arguments.
- **Strict Matching:** Every argument in the list **must** match the data type of the first non-null value found. If the list expects a Number and finds a Character, Oracle will throw ORA-00932: inconsistent datatypes.
- **Example:** SELECT NAME, COALESCE(MOBILE, OPHONE, RPHONE, 1800600300) FROM CONTACTNO;

3. Conditional Functions

These functions implement "if-then-else" logic directly within the SELECT statement.

DECODE Function

DECODE is Oracle-proprietary and uses a search/result pattern.

- **Syntax Logic:** DECODE(expression, search1, result1, search2, result2, ..., default)
- **Behavior:** If no match is found and no default is provided, it returns NULL.
- **Advanced Use:** You can concatenate columns within the expression to simulate multi-column conditions.
- *Example:* SELECT DECODE(DEPARTMENT_ID || MANAGER_ID, '50124', SALARY + 1500, '60103', SALARY - 500, SALARY) FROM EMPLOYEES;

CASE Expressions

Standardized across ANSI SQL, CASE is more flexible and readable than DECODE.

CASE I (Simple CASE)

Compares a single expression to multiple values.

- *Example:* CASE LENGTH(FIRST_NAME) WHEN 5 THEN SALARY + 500 WHEN 6 THEN SALARY + 750 ELSE SALARY END

CASE II (Searched CASE)

Allows for complex Boolean logic, including ranges and inequality operators.

- **Salary Increment Logic:**

4. Conversion Functions and 3-Argument Syntax

Conversion functions explicitly move data between Character, Number, and Date types.

- **TO_NUMBER:** Converts a string to a number for calculation or comparison.
- **TO_DATE:** Converts a string to a date object.

- **TO_CHAR:** Converts numbers or dates to strings for custom formatting.

Advanced 3-Argument Syntax

High-level mastery requires understanding that TO_NUMBER and TO_CHAR accept an optional third argument to override session NLS parameters within the query.

- **Syntax:** TO_NUMBER(value, format, nls_params)
- **Example:** SELECT TO_NUMBER('12#345~67', 'U99G000D09', 'NLS_DUAL_CURRENCY= NLS_NUMERIC_CHARACTERS=~#') FROM DUAL; This allows you to process foreign currency formats without altering the entire database session.

5. NLS Session Parameters and Dynamic Formatting

National Language Support (NLS) parameters dictate how Oracle handles symbols. View them via SELECT * FROM NLS_SESSION_PARAMETERS.

- **Key Parameters:** NLS_CURRENCY (Local symbol), NLS_NUMERIC_CHARACTERS (Decimal and Group separators), and NLS_DATE_FORMAT.
- **Modification:** Use ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.|';.
- **The Power of D and G:** Always use D (Decimal) and G (Group) in your format masks instead of hardcoded . or ,. D and G are dynamic; they update automatically when NLS settings change. Hardcoding these separators can lead to ORA-01481: invalid number format model.

6. Comprehensive Format Character Reference

Number Format Characters

Character,Meaning / Description
9,Displays a digit; returns a space if the digit is missing.
0,Displays a digit; returns a leading/trailing zero if missing.
". / ,",Hardcoded Decimal / Group separators (use with caution).
D / G,Dynamic Decimal / Group separators (references NLS settings).
$ / L,Default Currency ($) / Local Currency (defined by NLS_CURRENCY).
U,Dual/Euro Currency symbol (defined by NLS_DUAL_CURRENCY).
S,Displays a +/- sign (can be prefix or suffix).
MI,Displays a minus sign (-) as a suffix for negative values only.
PR,Wraps negative values in angle brackets < >.
EEEE,"Scientific notation/exponent format (e.g., 2.40E+04)."
V,Multiplies the value by 10 to the power of the number of digits after V.

Date Format Characters

Note: All format characters that return alphabetic values (MONTH, DAY, DY) are **case-sensitive** .| Category | Patterns | Description || ------ | ------ | ------ || **Day** | D, DD, DDD | Day of week (1-7), Day of month (1-31), Day of year (1-366). || | DY, DAY | Abbreviated (MON) or Full (MONDAY) name. || **Month** | MM, RM | Month number (01-12) or Roman numeral (I-XII).

|| | MON, MONTH | Abbreviated (JAN) or Full (JANUARY) name. || **Year** | Y, YY, YYYY | Last 1, 2, or 4 digits of the year. || | RR, RRRR | 2-digit or 4-digit "Round Year" logic. || | YEAR | Year spelled out (e.g., "TWO THOUSAND TWENTY-SIX"). || **Special** | TH, SP | Ordinal suffix (DDTH = 21st) or Spelled out (DDSP = TWENTY-ONE). || | FX, FM | Fixed Format (strict match) or Fill Mode (toggles padding). || | W, WW, Q, J | Week of month, Week of year, Quarter (1-4), Julian Day. || **Time** | HH, MI, SS | Hour (1-12), Minutes (0-59), Seconds (0-59). || | SSSSS | Total seconds past midnight (0-86399). |

7. Advanced Formatting Rules & Modifiers

Julian Date Depth

The Julian day (J) is a continuous count of days starting from **January 1, 4712 BC** .

- **Max Range:** Oracle supports Julian dates up to December 31, 9999 AD (Day 5373484).
- **The JSP Trick:** Use JSP to spell out any number. By converting a number to a Julian date and back to a character with the SP modifier, you can spell out any integer.
- *Example:* SELECT TO_CHAR(TO_DATE(1234, 'J'), 'JSP') FROM DUAL; returns "ONE THOUSAND TWO HUNDRED THIRTY-FOUR".

Century Logic: RR vs. YY

- **YY:** Assumes the current century. '95' becomes 2095.
- **RR:** Assumes the year is in the **1950–2049** window. If the current year is 2024, '95' is assumed to be 1995, while '05' is assumed to be 2005.

FM and FX Modifiers

- **FM (Fill Mode):** Character functions like MONTH pad results with spaces to match the length of the longest month (September). FM toggles these spaces and leading zeros off.
- **FX (Fixed Format):** Forces TO_DATE to require an exact match for punctuation and spacing. Using FX with a mismatched string results in ORA-01861.

8. Practice Examples and Syntax Summary

Professional Mastery Snippets

1. **Consonant Counting Logic:** To count consonants, use TRANSLATE to strip vowels and measure the remaining length.
2. **Left-Most Vowel Extraction:** Combine LTRIM (to remove leading consonants) and SUBSTR.
3. **Complex Nesting:** Displaying tenure with literal text.

Quick Reference: Common ORA Errors

- **ORA-01722 (Invalid Number):** Occurs when a string contains non-numeric symbols (like $) and is used in a calculation without a format mask.
- **ORA-01843 (Not a Valid Month):** Occurs when the month string doesn't match the NLS format or TO_DATE template.

- **ORA-01854 (Julian range error):** Julian dates must be between 1 and 5373484.
- **ORA-01861 (Literal mismatch):** Triggered when FX is used and the input punctuation doesn't match the template exactly.
- **ORA-01481 (Invalid format model):** Triggered when using invalid format characters or hardcoding separators that conflict with NLS expectations.